

Ch. 1 - Introduction

Secure software development and web security

R. Absil

Haute École Bruxelles-Brabant
École supérieure d'Informatique



September 18, 2025

Table of contents

- 1 Objectives
- 2 Cryptographic tools
 - Hash functions
 - Cipher algorithms
 - Signatures
- 3 Key exchange
 - Certificates and PKI
- 4 Some applications
 - TLS
 - End to end encryption
 - Threshold cryptography

Before we start

- Since WW2, there is a necessity to protect computer systems
 - Different types of threat
 - Different types of protection

Problem

- Strong security is heavier

Solution

- Analyse each risk and its probability to occur, and adapt your strategy

Types of risks

- Different types of risks
 - Access a restricted resource or service
 - Usurp somebody else's identity
 - Access confidential data
 - Falsification
- In this course, *only* these risks are covered
- For each of those risks
 - Several possible attacks
 - Several counter-measures available

Objectives

"When in doubt, use brute force"

- Many security systems rely on hard to solve mathematical problems
 - Hard: takes a "considerable" amount of time without the proper codes

Example

- Big numbers factorisation
 - Invert functions
 - Combinatorial optimisation
 - Stochastic optimisation
-
- We're interested in this type of constraints
 - Maths left aside

Objectives

- 1 Availability: make sure a service is available for authorized users
- 2 Authentication: make sure only authorised users have access to restricted resources and services
- 3 Non repudiation: make sure a contract between x and y *really* comes from x and is *really* being concluded with y
- 4 Confidentiality: make sure confidential data are confidential for anyone not authorised
- 5 Integrity: make sure data are left unaltered during transit

Objectives

- 1 Availability: make sure the system is up and running
- 2 Authentication: knowing “who is who”
- 3 Authorisation: knowing “who can do what”
- 4 Accountability: knowing “who did what”
- 5 Integrity: detect malicious modification of data
- 6 Confidentiality: sensitive data shouldn't be disclosed to anyone

Remark

- Authentication, authorisation and accountability are often abridged into AAA

Kerckoff's principles

Principles

- 1 The system must be materially and mathematically unbreakable without the key
 - 2 The system must not require any secret (beyond the key), and must not be compromised should it fall into enemy hands
 - 3 The system must be “easy” to use
-
- Security *cannot* rely on secrecy
 - The opponent *knows* the cryptographic / security system
 - A system's security must rely on the key (or access codes) *only*, not on the system itself

Transparency and delegation

- A developer *should* make software safe without both asking for it and knowing how it works
 - Ex.: Framework-provided password-based authentication
 - Ex.: TLS tunnels from frontend to backend
- Do not implement from scratch robust complex mechanisms
 - Delegate to a known working and safe solution
 - Ex.: OAuth authentication delegation

Idea

- You can't make wrong something that is made right for you
 - ... especially when you don't know what you are protected against

Cryptographic tools

Table of contents

1 Objectives

2 Cryptographic tools

- Hash functions
- Cipher algorithms
- Signatures

3 Key exchange

- Certificates and PKI

4 Some applications

- TLS
- End to end encryption
- Threshold cryptography

Cryptographic hash functions

Definition

- Let $f : M \mapsto \mathbb{N}$, f is a cryptographic hash function if
 - 1 computing $f(x)$ is easy for each $x \in M$
 - 2 f is resistant to pre-image
 - it is hard to invert f ,
 - 3 f is resistant to second pre-image
 - it is hard to change a message without its hash
 - 4 f is resistant to collisions
 - it is hard to generate two different messages with the same hash
- Hash functions are also public
 - No secret involved, such as a key
- The output as fixed (bounded) size
 - Makes practical uses easy

Illustration: checking integrity

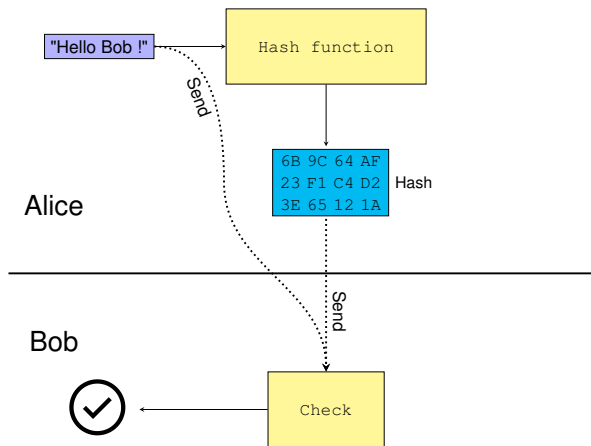


Table of contents

1 Objectives

2 Cryptographic tools

- Hash functions
- Cipher algorithms
- Signatures

3 Key exchange

- Certificates and PKI

4 Some applications

- TLS
- End to end encryption
- Threshold cryptography

Cipher algorithms

Objective

- Encode messages with one or two “keys”
 - 1 Single key: symmetric cryptography
 - This key is *secret*
 - 2 A key to encode (public), a key to decode (private): public key (asymmetric) cryptography
 - Only the private key is secret
 - The operation that needs to be protected is performed with the private key
 - Encryption: we cipher with the public key, and decipher with the private one
- Without the (private) key, it is hard to recover the original message
 - Or to deduce the private key from the public one
- Different from steganography

Illustration: symmetric cipher

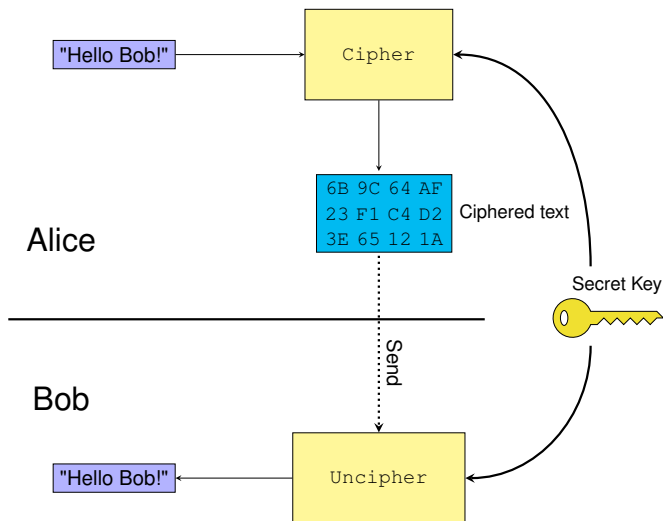


Illustration: public key cipher

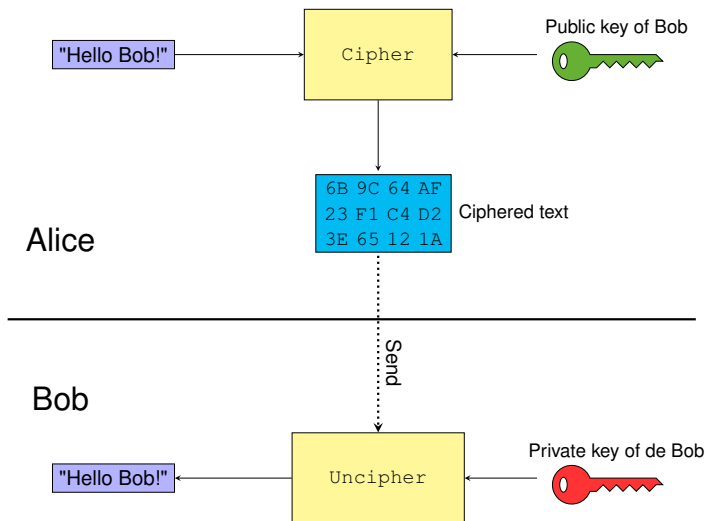


Table of contents

1 Objectives

2 Cryptographic tools

- Hash functions
- Cipher algorithms
- **Signatures**

3 Key exchange

- Certificates and PKI

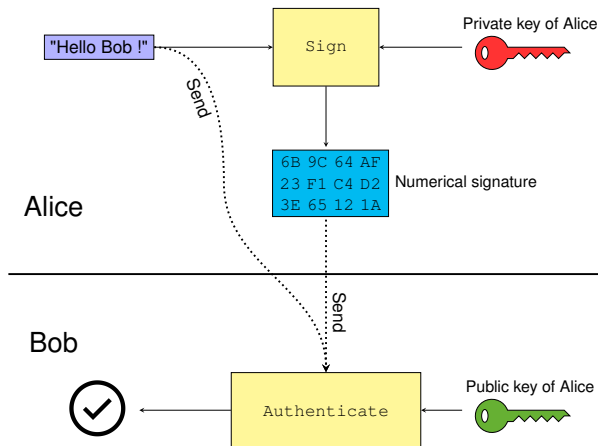
4 Some applications

- TLS
- End to end encryption
- Threshold cryptography

Signature algorithms

- Basic idea: make sure the sender is who we think he is
- More sophisticated than a signature at the bottom of a page
 - Usurp the identity of someone must be hard
 - Principle: we sign “through” the whole document, imprinting data.
- Works with a combination of hash functions and ciphers
 - We cipher the hash of the message with our private key
 - The signature is authenticated with our public key
- Makes sure the message
 - was left unaltered
 - is genuine

Illustration: signature



Key exchange

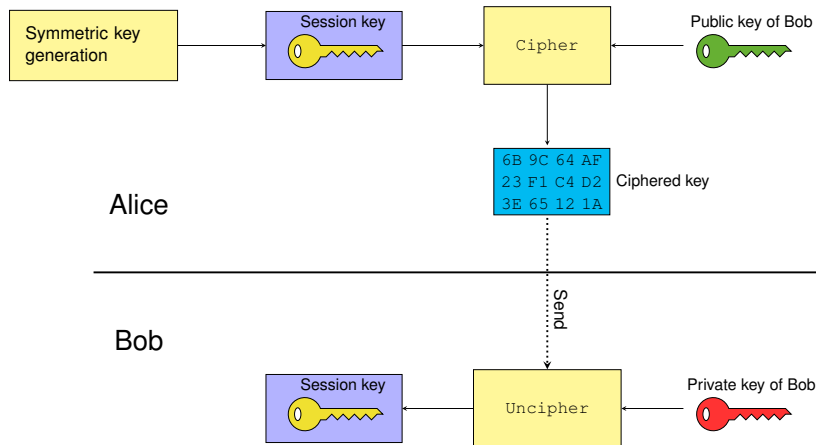
The context

- If two actors never met before, they have to exchange keys
 - to encrypt sensitive information they send to each others
 - to sign messages
- Symmetric cryptography is faster than public key cryptography
 - We would rather use it

Problems

- Symmetric keys cannot be intercepted
- We don't have any information about the *identity* of the owner of a public key

Illustration: RSA key exchange



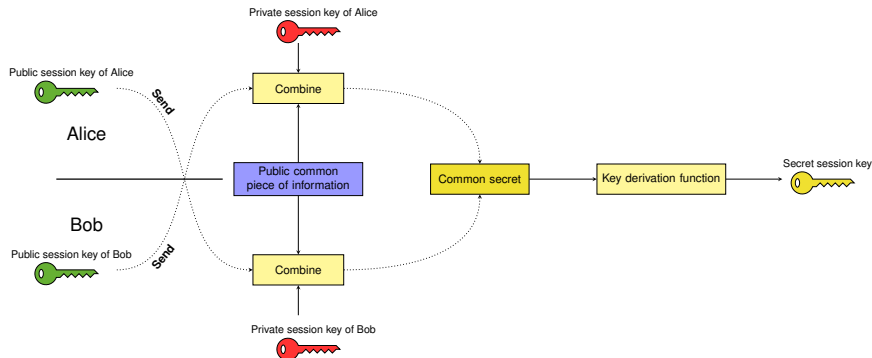
Forward secrecy

- We use a single pair of private/public keys during several sets of transactions

What if the private key is compromised?

- An attacker can decrypt all previously performed transactions
- Private keys used for encryption have to be renewed with each connection
 - Not the ones used for authentication
- This way, if a key is compromised, previous transactions cannot be decrypted
- The Diffie-Hellman key exchange (and variants) have this property
 - Each actor combines his private key with the public key of his correspondent
 - Safely produces a common secret used to generate keys

Illustration: DHKE



- The “combine” algorithm is chosen in such a way that it has the *same* output when executed with
 - the public key of Bob and the private key of Alice,
 - the private key of Bob and the public key of Alice.

Table of contents

- 1 Objectives
- 2 Cryptographic tools
 - Hash functions
 - Cipher algorithms
 - Signatures
- 3 Key exchange
 - Certificates and PKI
- 4 Some applications
 - TLS
 - End to end encryption
 - Threshold cryptography

Certificate

- Electronic document proving the ownership of a public key
 - Issued/revoked/renewed by a certification authority (CA) according to the rules of a PKI
 - A registration authority checks the identity of the owner
- It is assumed involved actors *both* trust the CA
 - With the help of a chain of trust
 - Act as a trusted third party intermediated
- Contains
 - information about the public key
 - information about the owner's identification
 - numerical signature of the issuer
 - serial number, period of validity, etc.
- The CA makes sure information stored in the certificate are correct, and signs it
 - Usually, a user crafts a CSR, signs it, and sends it to his CA in order to craft a certificate

Chain of trust

- It is difficult, in practice, to have only a few CA
 - Lots of different CA, with different price ranges, different requirements, etc.

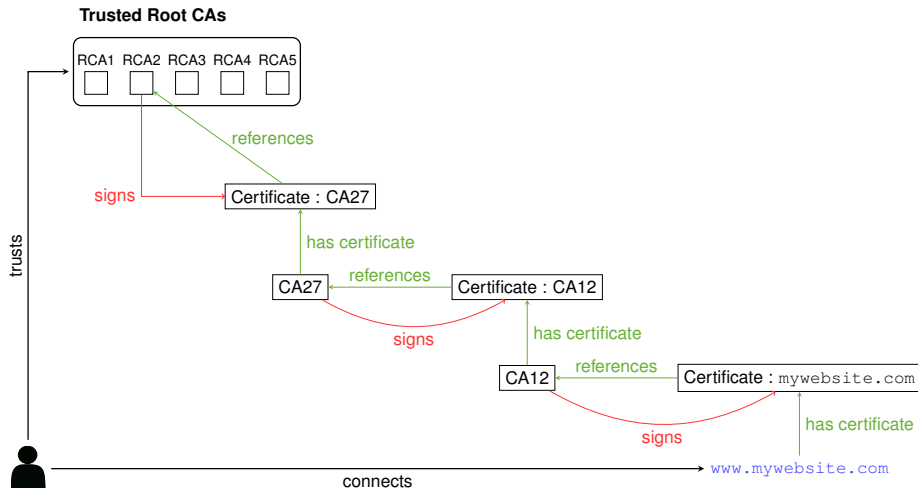
Question

- How can a certificate issued by some CA recognised by Apple can be trusted on somebody running a Microsoft software?

Answer

- Apple and Microsoft and a third party intermediate that they both trust
 - That CA signs both Microsoft and Apple certificates
 - Such “high level” CAs are called *root* CAs
 - Web browsers pack up lists of root CAs they trust

Illustration



Some applications

Overview

- Cryptographic primitives are widely used in cybersecurity
 - Security relies on their properties
 - Hard to invert a hash function, decipher without the private key, etc.
- We overview a few of them and emphasise on the properties the security system has
 - TLS: widely used in tunnel-requiring connections (Https, VPNs, RADIUS, etc.)
 - Sequences of items: checking they cannot be forged, and that the sequence has integrity
 - End to end encryption: how to store and transmit to servers that you cannot trust

Table of contents

- 1 Objectives
- 2 Cryptographic tools
 - Hash functions
 - Cipher algorithms
 - Signatures
- 3 Key exchange
 - Certificates and PKI
- 4 Some applications
 - TLS
 - End to end encryption
 - Threshold cryptography

Objectives

- When you connect to a server, you want
 - to actually connect to the actual server,
 - the privacy and integrity of exchanged data.
- Protection against MITM attacks, replay attacks, etc.
- Bidirectional encryption between client and server
- Enabled with certificates
 - The server is authenticated
- The first step of the connection is to perform a TLS handshake

Objective

- Securely generate and share a session key using DHKE
- That key will be used to encrypt data in both ways

Step-by-step in TLS 1.3

1 Client Hello

- The client generates a random key pair for a given algorithm, and sends a list of supported ciphers, a random nonce, and the generated public key

2 Server Hello

- If the server supports the algorithm the client guessed, it generates a key pair, and sends the public key it generated and a random nonce (both signed), and its certificate

3 Client Finish

- 1 The client checks the certificate with its CA, and the signature
- 2 The client generates a pre-master secret, encrypt it with the server public key, and sends it
- 3 The client generates a session key with DH

4 Server Finish

- 1 The server receives the pre-master secret from the client, and decrypts it
- 2 The server generates a session key with DH

Table of contents

- 1 Objectives
- 2 Cryptographic tools
 - Hash functions
 - Cipher algorithms
 - Signatures
- 3 Key exchange
 - Certificates and PKI
- 4 Some applications
 - TLS
 - End to end encryption
 - Threshold cryptography

Why

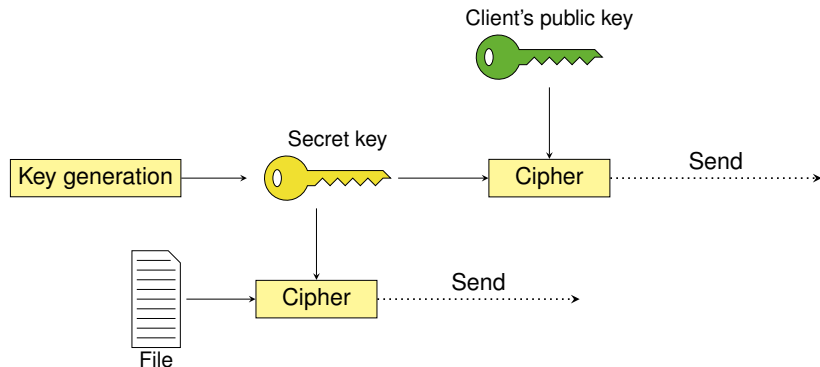
- On tunnel-relying protocols, data is automatically encrypted and decrypted by clients *and* servers
 - Such as with TLS, VPNs, etc.
- Servers have the ability to record and monitor everything
 - ... which doesn't mean they do
- The same “problem” happens with database autociphering
 - Entries are automatically encrypted / decrypted
- Protocols usually do not guarantee the confidentiality of metadata
- If you can't trust a server, you need a solution
 - For instance, to store sensitive informations on a server

Principle

End-to-end encryption

- Data is encrypted on client-side, by the sender's device
- Cryptographic material is also generated on client-side
 - ... but can still be stored *securely* on server side
- Since cryptographic material is generated on client-side, the server doesn't have access to it
 - The server cannot use your key to decipher
- Since data is encrypted on client-side, the server never has it plain
- Nobody except the sender has the ability to decrypt

Example: file storage with a keypair



- These steps happen on client side
- The server can't decipher, because it doesn't have the private key

Table of contents

- 1 Objectives
- 2 Cryptographic tools
 - Hash functions
 - Cipher algorithms
 - Signatures
- 3 Key exchange
 - Certificates and PKI
- 4 Some applications
 - TLS
 - End to end encryption
 - Threshold cryptography

Threshold cryptography

- Idea: we want to share a secret
 - Cryptographic key for ciphering or signing
 - Access codes
- Motivation: allow service if at least x people agree to cooperate-
 - Bank safe, missile codes, etc.

Overview

Basic idea

- We want to share a secret (a key) between several people
 - We want to be able to recover the secret if *enough* of these people cooperate
 - If an insufficient number of people is gathered, the secret is safe
-
- How to proceed ?!
 - Several (mathematical) techniques
 - Here: polynomial interpolation
 - Motivation
 - Fight against corruption of actors
 - Force several people to cooperate to make important decisions

(k, n) threshold cryptography

Definition

- A cryptographic system is called a *threshold cryptographic system* if several people (more than the threshold) need to cooperate in the protocol used to (de)cipher a message.
- Such a system is a (k, n) -threshold if at least k people among n must cooperate to (de)cipher the message
- Idea: the private key is shared among actors
- Independently discovered by Shamir (polynomials) and Blakley (hyperplanes) in 1979

Key sharing (1/2)

Safe sharing

- *Safe sharing* if someone with less than k pieces as no more information that someone with no pieces.
- Example: secret Password to share
 - 1 People 1: Pa
 - 2 People 2: . . ss
 - 3 People 3: wo . .
 - 4 People 4: rd
- Someone with no pieces must guess among $26^8 \simeq 208$ billions possibilities
- Someone with one piece must guess among $26^6 \simeq 308$ millions possibilities
- Unsafe sharing

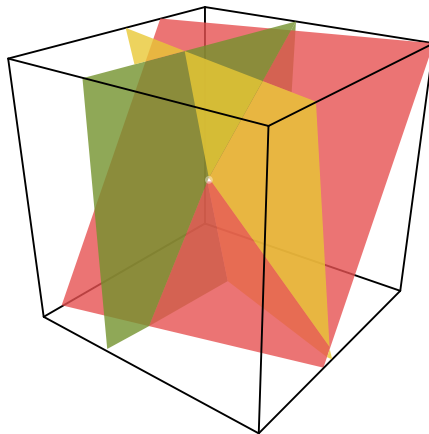
Key sharing (2/2)

- A secret is iteratively ciphered with k public keys
- The k corresponding private keys are distributed among $n = k$ actors
 - Someone can remove the first level of cipher
 - If two people cooperate, they can remove two levels
 - Unless k persons cooperate, the secret is still completely confidential
- Bypassing the system without k keys is reduced to decipher datas without a key
 - Not very practical, but safe

Problem

- How to proceed so that $k < n$ people are able to decipher ?

Example with few people (Blakley)



Polynomial interpolation

- In the case of Shamir secret sharing, we rely on polynomial interpolation
- To share a secret among k people,
 - 1 we work with a polynomial of degree $(k - 1)$
 - 2 The key is one of the coefficient (arbitrary)
 - 3 Other coefficients are generated at random
 - 4 We generate n distinct points of the polynomial and give one of them to each actor
- If k actors cooperate, we have k points of a polynomial of degree $k - 1$
 - 1 We build it with polynomial interpolation
 - 2 We recover the key

In practice

- We have $n + 1$ points $p_i = (x_i, y_i)$
- We want to compute the coefficients of a polynomial

$$\begin{aligned}
 P(x) &= \sum_{i=0}^n a_i x^i \\
 &= a_n x^n + a_{n-1} x^{n-1} + \dots + a_2 x^2 + a_1 x + a_0
 \end{aligned}$$

- Substituting x in this equation and $P(x)$ by their values, we get a system with $n + 1$ equations and $n + 1$ unknown (the coefficients)

$$\begin{bmatrix} x_0^n & x_0^{n-1} & x_0^{n-2} & \dots & x_0 & 1 \\ x_1^n & x_1^{n-1} & x_1^{n-2} & \dots & x_1 & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ x_n^n & x_n^{n-1} & x_n^{n-2} & \dots & x_n & 1 \end{bmatrix} \begin{bmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_0 \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

System solving in practice

- We must solve this system in the variables a_i
- Mimic the "manual" method
 - Make the matrix triangular
- Gaussian method
 - Running time: $\mathcal{O}(n^3)$
 - High conditioning
 - Subject to numerical errors: numerical instability
 - How a controlled error on inputs alters the output ?
- Other methods
 - Rewrite the polynomial
 - Lagrangian interpolation
 - Newtonian interpolation
 - Divided differences
- Use Lapack¹ for anything related to linear algebra

¹<https://netlib.org/lapack/>